Solution: Quick Sort

Problem Statement

Write Recursive Approach for Quick Sort

Given an array of integers, sort it in ascending order using the Quick Sort algorithm.

Examples

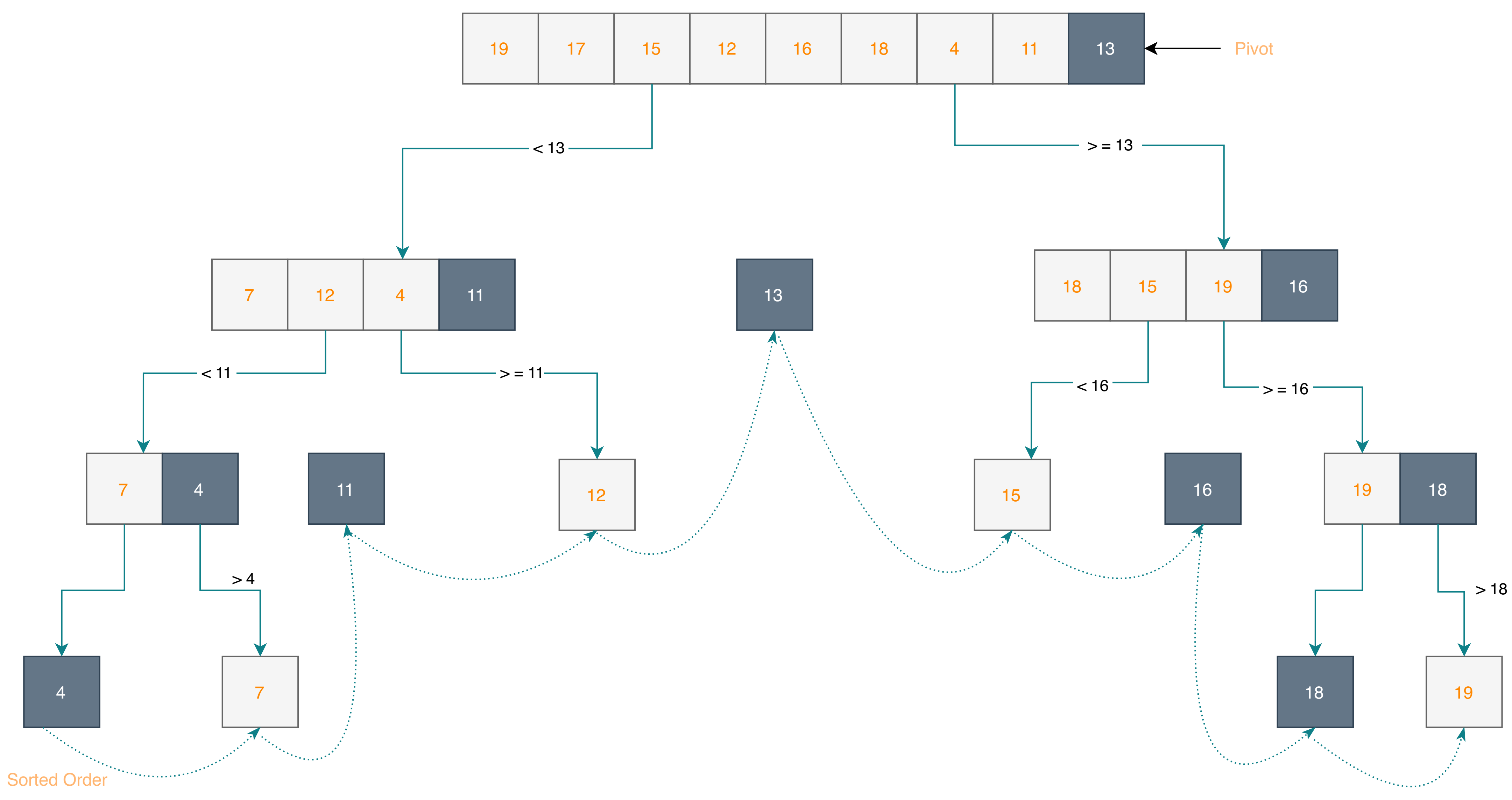
Sr#	Input Array	Output	Description
1	[4, 2, 6, 8, 3]	[2, 3, 4, 6, 8]	The array is sorted in ascending order.
2	[10, 5, 3, 7, 2, 8, 6]	[2, 3, 5, 6, 7, 8, 10]	The array is sorted in ascending order.
3	[1, 2, 3, 4, 5]	[1, 2, 3, 4, 5]	The array is already sorted in ascending order.

Solution

The Quick Sort algorithm follows a divide-and-conquer approach to sort an array. It works by selecting a pivot element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then recursively sorted. The base case of the recursion is an array with zero or one element, which is already considered sorted.

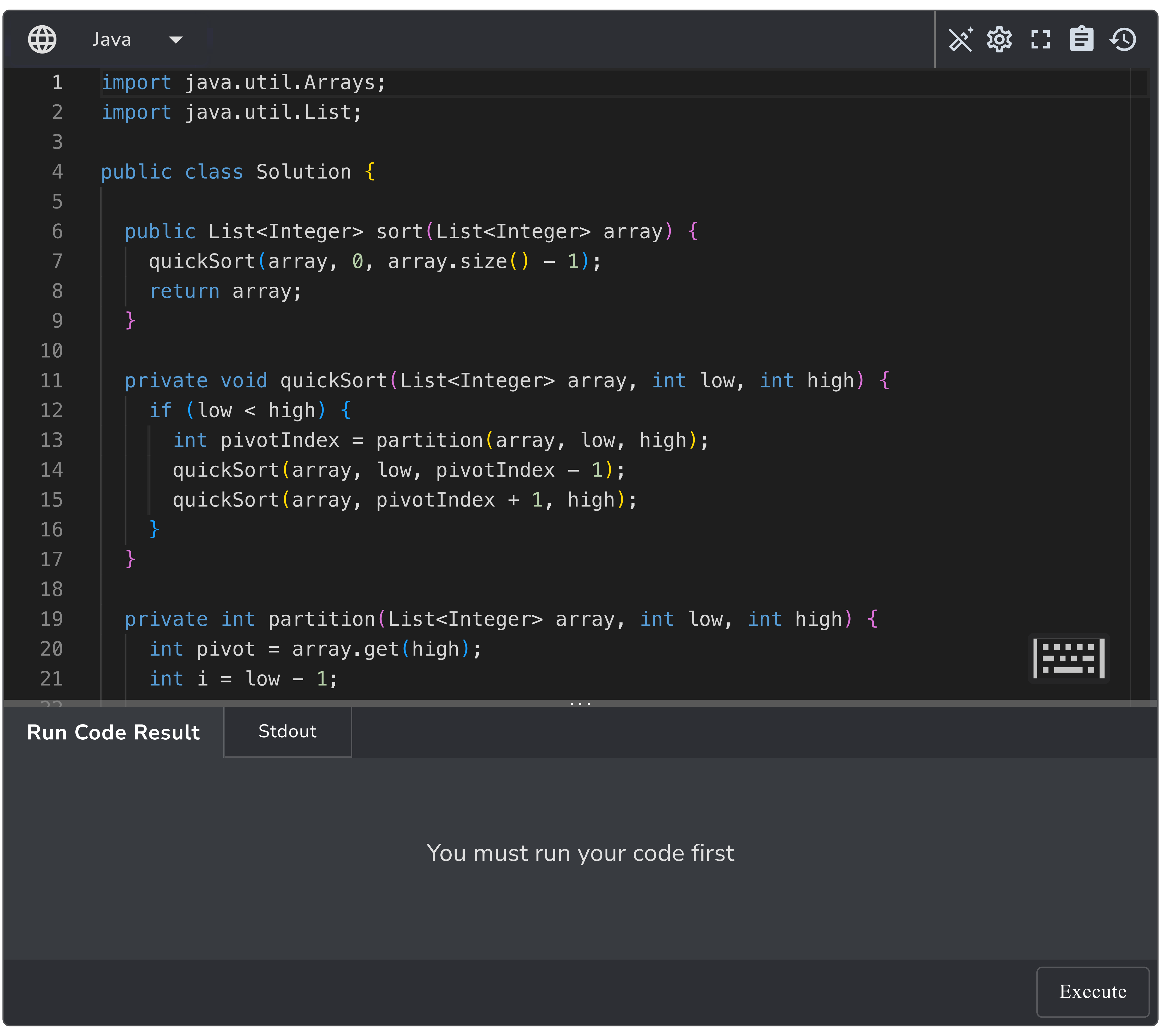
The steps of the Quick Sort algorithm can be summarized as follows:

1. Select a pivot element from the array.
2. Partition the array into two sub-arrays: one with elements less than the pivot and one with elements greater than the pivot.
3. Recursively apply Quick Sort to the sub-arrays.
4. Combine the sorted sub-arrays and the pivot element to obtain the final sorted array.



Code

Here is the code for this algorithm:



```java
import java.util.Arrays;
import java.util.List;

public class Solution {

  public List<Integer> sort(List<Integer> array) {
    quickSort(array, 0, array.size() - 1);
    return array;
  }

  private void quickSort(List<Integer> array, int low, int high) {
    if (low < high) {
      int pivotIndex = partition(array, low, high);
      quickSort(array, low, pivotIndex - 1);
      quickSort(array, pivotIndex + 1, high);
    }
  }

  private int partition(List<Integer> array, int low, int high) {
    int pivot = array.get(high);
    int i = low - 1;
```

You must run your code first

Time and Space Complexity

The time complexity of the Quick Sort algorithm is $O(n * logn)$ on average and in the best case, where n is the number of elements in the array. This is because the algorithm divides the array into smaller sub-arrays and recursively sorts them. The worst-case time complexity is $O(n^2)$ when the array is already sorted or almost sorted, and the pivot is chosen poorly. However, the probability of encountering the worst case is low.

The space complexity of the algorithm is $O(logN)$ on average and in the best case, corresponding to the recursive call stack. In the worst case, the space complexity can be $O(N)$ due to the partitioning process. However, the overall space usage is efficient as the algorithm performs in-place sorting by swapping elements within the array.

← Previous
13. Quick Sort

Next →
14. Depth First Search

☐ Mark as Completed